

KANDIDATNUMMER(E)/NAVN:

Ottersbo, Elias Alexander Wiklund (10072)
Evensen, Sigurd (10076)

DATO:

26/5 - 2026

FAGKODE:

IDATT2003

STUDIUM:

BIDATA - TRONDHEIM

ANT SIDER/BILAG:

24 /

FAGLÆRER(E) :

- Atle Olsø
- Majid Rouhani

TITTEL :

IDATT2003 Programmering 2 – Mappevurdering 2026 Rapport

SAMMENDRAG:

Denne rapporten dokumenterer design og utvikling av Millions, et aksjemarkedssimuleringsspill utviklet som en del av mappeoppgaven i IDATT2003 Programmering 2. Applikasjonen lar brukeren søke etter aksjer, kjøpe og selge andeler, følge utviklingen i porteføljen over tid, og lagre og laste inn spilltilstand. Løsningen er implementert som en desktop-applikasjon med grafisk brukergrensesnitt bygget i JavaFX, mens kjernefunksjonaliteten er organisert i egne modell-, kontroller- og infrastrukturelag.

Prosjektet legger vekt på objektorientert design, lagdelt arkitektur, robust validering, persistens med SQLite og automatisert testing med JUnit. Rapporten beskriver kravgrunnlaget, teoretisk fundament, utviklingsprosess, teknisk løsning, teststrategi og en drøfting av både produktet og arbeidsprosessen.

Denne oppgaven er en besvarelse utført av student(er) ved NTNU.

Deklarasjon om KI-hjelpemidler

Har det i utarbeidingen av denne rapporten blitt anvendt KI-baserte hjelpemidler?

☐ Nei

☒ Ja

Hvis *ja*: spesifiser type av verktøy og bruksområde under.

Tekst

☐

Stavekontroll. Er deler av teksten kontrollert av:
Grammarly, Ginger, Grammarbot, LanguageTool, ProWritingAid, Sapling, Trinkai eller lignende verktøy?

☐

Tekstgenerering. Er deler av teksten generert av:
ChatGPT, GrammarlyGO, Copy.AI, WordAi, WriteSonic, Jasper, Simplified, Rytr eller lignende verktøy?

☐

Skriveassistanse. Er en eller flere av ideene eller fremgangsmåtene i oppgaven foreslått av:
ChatGPT, Google Bard, Bing chat, YouChat eller lignende verktøy?

Hvis *ja* til anvendelse av et tekstverktøy - spesifiser bruken her:

Kode og algoritmer

☐

Programmeringsassistanse. Er deler av koden/algoritmene som i) fremtrer direkte i rapporten eller ii) har blitt anvendt for produksjon av resultater slik som figurer, tabeller eller tallverdier blitt generert av: *GitHub Copilot, CodeGPT, Google Codey/Studio Bot, Replit Ghostwriter, Amazon CodeWhisperer, GPT Engineer, ChatGPT, Google Bard* eller lignende verktøy?

Hvis *ja* til anvendelse av et programmeringsverktøy - spesifiser bruken her:

Bilder og figurer

☐

Bildegenerering. Er ett eller flere av bildene/figurene i rapporten blitt generert av: *Midjourney, Jasper, WriteSonic, Stability AI, Dall-E* eller lignende verktøy?

Hvis *ja* til anvendelse av et bildeverktøy - spesifiser bruken her:

Andre KI-verktøy

☒

Andre KI-verktøy. har andre typer av verktøy blitt anvendt? Hvis *ja* spesifiser bruken her:

Se kapittel 3.3.

☒

X

Jeg er kjent med NTNUs regelverk: *Det er ikke tillatt å generere besvarelse ved hjelp av kunstig intelligens og levere den helt eller delvis som egen besvarelse.* Jeg har derfor redegjort for all anvendelse av kunstig intelligens enten i) direkte i rapporten eller ii) i dette skjemaet

(KOMMER SENERE)

Underskrift/Dato/Sted

INNHold

Innholdsfortegnelse

1	Introduksjon	1
1.1	Bakgrunn.....	1
1.2	Kravspesifikasjon	1
1.3	Avgrensninger	4
1.4	Begreper/Ordliste	4
2	Teori.....	6
3	Metode	8
3.1	Utviklingsprosess.....	8
3.2	Verktøy	9
3.3	Bruk av KI verktøy.....	9
4	Resultat	10
4.1	Teknisk Design.....	10
4.2	Implementasjon.....	10
4.3	Testing.....	11
4.4	Utrulling til sluttbruker (deployment)	13
5	Drøfting	13
5.1	Drøfting av løsning/design	14
5.2	Drøfting av prosess.....	15
5.3	Drøfting av bruken av KI-verktøy	15
6	Konklusjon - erfaring.....	16

Figurliste

Figur 1	Use Case diagram	3
Figur 2	Klassediagram som viser... ..	11

Tabelliste

Tabell 1	Begreper og ordliste	5
----------	----------------------------	---

*[Denne rapporten inneholder ferdigdefinerte **stiler** som du/dere kan benytte for de mest vanlige avsnittene. Følgende stiler er definert:*

Heading 1/Overskrift 1 Overskrift på nivå 1

Heading 2/Overskrift 2 Overskrift på nivå 2

Heading 3/Overskrift 3 Overskrift på nivå 3

Brødtekst Standard tekst i et avsnitt. Benytt denne for all "vanlig" tekst

Definition Benyttes hovedsakelig i avsnittet "TERMINOLOGI"

References Benyttes i REFERANSER-avsnittet.

Comment Denne grønne teksten. Fjern all tekst av denne typen i rapporten.]

1 INTRODUKSJON

1.1 Bakgrunn

[Dette er første kapitlet i den faglige rapporten. Det bør behandle bakgrunnen for oppgaven, eventuell oppdragsgiver, oppsummering av problemstillingen og/eller oppgaven som skal løses.]

Dette prosjektet omhandler utviklingen av et aksjemarkedssimuleringsspill kalt Millions, utviklet som en del av mappeoppgaven i emnet IDATT2003 Programmering 2. Formålet med applikasjonen er å simulere handel i et forenklet aksjemarked, der brukeren kan kjøpe og selge aksjer, følge utviklingen i egen portefølje, se transaksjonshistorikk og observere hvordan aksjepriser endrer seg over tid.

I motsetning til en mindre, rent algoritmisk oppgave, krevde dette prosjektet utvikling av en helhetlig programvareløsning med både grafisk brukergrensesnitt og en intern arkitektur som håndterer domenelogikk, validering, lagring og administrasjon av tilstand. Prosjektet kombinerer dermed praktisk programvareutvikling med objektorientert design, brukerinteraksjon og fokus på vedlikeholdbar kode.

Den endelige løsningen er implementert som en Java-basert desktop-applikasjon med JavaFX som brukergrensesnitt. Programmet laster aksjedata fra CSV-filer, simulerer ukentlige endringer i markedet, lar brukeren gjennomføre handler, og lagrer spilltilstanden ved hjelp av en SQLite-basert persistensløsning. På denne måten går prosjektet utover en enkel prototype og fremstår som et mer helhetlig programvareprodukt.

1.2 Kravspesifikasjon

*[Her beskriver du både de **funksjonelle** kravene og de **ikke-funksjonelle** kravene til løsningen du skal utvikle.*

Dersom det er gitt en kravspesifikasjon vil du kunne hente det meste av informasjon fra denne. Husk at du her IKKE skal beskrive noen av de valg du har gjort i prosjektet, eller det du konkret har utviklet i prosjektet.

*Bruk her gjerne **UML-diagrammer** som **Use-Case**, **Aktivitetsdiagram** osv for å beskrive krav til funksjonalitet (NB! Uten å dra inn **hvordan** du/dere har løst det.).*

*Når du senere skriver **drøfting** og **konklusjon**, må du henvise tilbake til dette kapittelet og svare på om løsningen du har levert svarer på kravspesifikasjonen].*

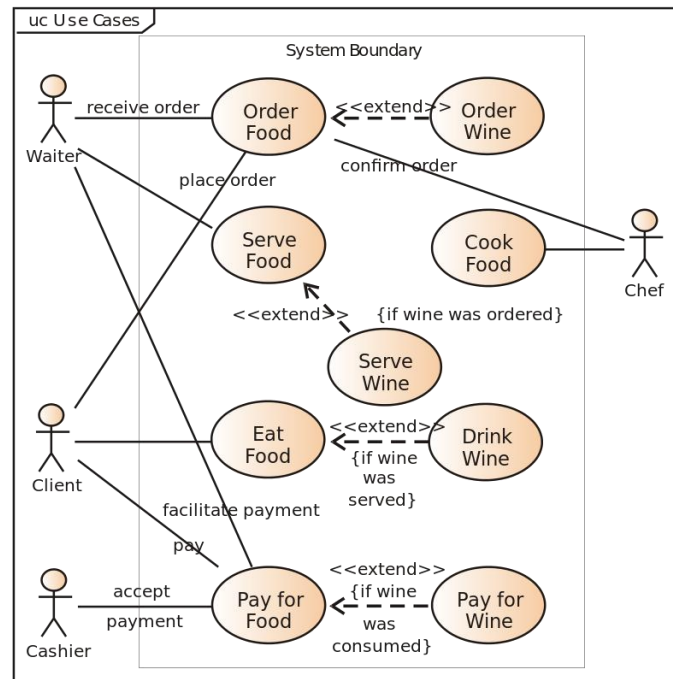
Oppgaven krever utvikling av en fungerende programvareløsning i Java som demonstrerer god objektorientert struktur, kodekvalitet og bruk av relevante programvaretekniske prinsipper. Basert på implementasjonen i repositoryet er følgende funksjonelle krav sentrale i løsningen:

- systemet skal la brukeren starte et nytt spill med valgt navn og startkapital
- brukeren skal kunne se tilgjengelige aksjer i markedet
- brukeren skal kunne søke etter aksjer basert på symbol eller selskapsnavn
- brukeren skal kunne kjøpe aksjer
- brukeren skal kunne selge aksjer som eies
- systemet skal holde oversikt over brukerens portefølje
- systemet skal lagre historikk over gjennomførte transaksjoner
- systemet skal simulere tidsprogresjon i form av uker
- aksjepriser skal endres over tid
- brukeren skal kunne lagre og laste inn spilltilstand
- løsningen skal presenteres gjennom et grafisk brukergrensesnitt

I tillegg til de funksjonelle kravene finnes det flere ikke-funksjonelle krav som er viktige for kvaliteten på løsningen:

- løsningen bør være modulær og vedlikeholdbar
- implementasjonen skal demonstrere objektorienterte designprinsipper
- koden skal inneholde tydelig validering og feilbehandling
- persistensløsningen skal være strukturert og robust
- testing skal brukes for å verifisere sentrale deler av systemet
- applikasjonen skal være forståelig og brukervennlig

Et viktig kvalitetskriterium for en sterk besvarelse er ikke bare at applikasjonen fungerer, men også at designvalgene kan begrunnes og knyttes til gode programvaretekniske prinsipper. Derfor drøfter rapporten løsningen opp mot blant annet kobling, samhörighet, ansvarsdeling, lagdelt arkitektur, validering og persistensdesign.



1.3 Avgrensninger

*[Er det gitt noen avgrensninger/begrensninger i oppgaven? Beskriv i så fall disse her. Avgrensninger kan f.eks. være teknologier dere **må** bruke, el.l. Dersom prosjektet ikke har noen avgrensninger, kan dette kapittelet utelates.]*

Prosjektet har flere naturlige avgrensninger, både som følge av oppgavens omfang og valg som er gjort under utviklingen.

For det første er systemet en simulering, ikke en reell handelsplattform. Aksjeprisene genereres og oppdateres internt i systemet, i stedet for å hentes fra eksterne markedsdata eller sanntids-API-er. Applikasjonen legger derfor større vekt på modellering, struktur og brukerinteraksjon enn på finansiell realisme.

For det andre er persistensløsningen lokal. Lagrede spill lagres i SQLite på brukerens egen maskin, og systemet støtter verken skybasert lagring, autentisering eller flerbrukerfunksjonalitet.

For det tredje er brukergrensesnittet, selv om det er grafisk og mer brukervennlig enn en konsollapplikasjon, fortsatt begrenset til det som er hensiktsmessig innenfor rammene av et emneprosjekt. Funksjoner som avanserte grafer, pålogging, nettverksstøtte eller mer realistiske markedsmekanismer er bevisst utelatt for å kunne fokusere på kjernefunksjonalitet og kodekvalitet.

Til slutt ser enkelte implementasjonsvalg ut til å være påvirket av spesifikasjonen heller enn av ideell domenemodellering. Ett eksempel er bruk av **BigDecimal** for aksjeantall, noe som også kommenteres i kodebasen som et litt unaturlig valg i en realistisk løsning, men som likevel er fulgt for å samsvare med oppgavens premisser.

1.4 Begreper/Ordliste

[Når man utvikler programvare for en kunde, er det viktig å etablere en felles forståelse for begreper/terminologi/ord som benyttes **av kunden**. Det er derfor svært vanlig å lage en «ordliste» og/eller en «Domene modell». Denne ordlisten er også et svært godt utgangspunkt for å finne frem til hvilke mulige **klasser** det kan være aktuelt å implementere i løsningen når denne skal utvikles. Bruk tid på denne slik at du har en god forståelse for begrepene.

Her kan man også bruke klasse-diagrammer for å illustrere hvordan **begreper** henger sammen. NB! Klasser i dette diagrammet er **ikke** klassene du har valgt å implementere i prosjektet.]

Begrep (Norsk)	Begrep (Engelsk)	Betyding/beskrivelse
Aksje	Stock	Et omsettelig finansielt objekt med symbol, selskapsnavn og prishistorikk.
Aksjepost	Share / Share holding	En beholdning av et gitt antall aksjer i ett selskap, med tilhørende kjøpspris.
Portefølje	Portfolio	Samlingen av aksjer som brukeren eier.
Børs	Exchange	Markedet hvor aksjene er registrert og handles.
Transaksjon	Transaction	En gjennomført kjøps- eller salgsoperasjon.
Kjøp	Purchase	En transaksjon hvor brukeren kjøper aksjer og betaler pris pluss gebyr.
Salg	Sale	En transaksjon hvor brukeren selger aksjer og mottar netto beløp etter gebyr og eventuell skatt.
Spilltilstand	Game state	Den samlede tilstanden til spillet, inkludert børs og spillerdata.
Persistens	Persistence	Mekanismen for å lagre og laste spilldata.
Markedsbevegelse	Market movement	Ukentlig prisendring på aksjer i simuleringen.
Startkapital	Starting capital	Beløpet brukeren starter spillet med.
Transaksjonsarkiv	Transaction archive	Historikk over gjennomførte handler.

Tabell 1 Begreper og ordliste

2 TEORI

*[Her beskriver du de **teoriene** og **beste praksiser** som dere har benyttet ved utvikling av en programvare, som kobling, cohesion, SOLID osv. Beskriv **hvilke teorier** og **IKKE hvordan** dere har brukt teoriene. Det hører hjemme i resultat og diskusjon.*

*Her må dere referere til **kilder** (som legges til i kapittelet **Referanser**).*

Når du senere skal skrive drøfting og konklusjon, må du henvise tilbake til dette kapittelet og svare på om løsningen du har levert er løst i henhold til disse teoriene/beste praksisene.

Unngå å beskrive om grunnleggende teorier som «hva er OOP», «Programmeringsspråket Java» osv.

*Skriv **kun teoripunkter som du faktisk har benyttet i ditt design, og som du faktisk diskuterer i diskusjonen** 😊]*

2.1 Objektorientert design

Et sentralt mål i objektorientert utvikling er å modellere problemdomenet gjennom klasser med tydelige ansvarsområder og meningsfulle relasjoner. I et godt strukturert objektorientert system representerer hver klasse et begrep eller ansvar i domenet, og samspillet mellom objektene spiller hvordan systemet faktisk fungerer.

I dette prosjektet er begreper som **Stock**, **Share**, **Player**, **Portfolio**, **Transaction** og **Exchange** modellert som egne domeneklasser. Dette styrker lesbarheten, gjør arkitekturen lettere å forstå, og gir en løsning der oppførsel i større grad plasseres nær dataene den opererer på.

2.2 Single Responsibility Principle

Single Responsibility Principle innebærer at en klasse bør ha ett hovedansvar, eller én primær grunn til å endres. Dette betyr ikke at en klasse bare kan inneholde én metode, men at metodene bør støtte opp om ett klart formål.

Dette prinsippet er særlig viktig i systemer som kombinerer flere hensyn, som domenelogikk, persistens og brukergrensesnitt. Hvis disse blandes for mye sammen, blir løsningen vanskeligere å vedlikeholde, utvide og teste. Klasser med tydelig ansvar gjør systemet enklere å forstå og reduserer risikoen for at endringer i én del skaper feil i en annen.

2.3 Lav kobling og høy samhörighet

Lav kobling betyr at ulike deler av systemet skal være så lite avhengige av hverandre som mulig. Høy samhörighet betyr at elementene innad i en klasse eller modul bør høre naturlig sammen og støtte samme ansvar.

Disse prinsippene er viktige for vedlikeholdbarhet og testbarhet. En klasse med høy samhörighet er lettere å forstå fordi den har et tydelig formål. Et system med lav kobling er enklere å videreutvikle fordi endringer i én klasse i mindre grad tvinger frem endringer andre steder.

2.4 Separation of concerns og lagdelt arkitektur

Separation of Concerns handler om å dele systemet inn i deler som håndterer ulike typer ansvar, som for eksempel domenelogikk, presentasjon og lagring. Lagdelt arkitektur er en praktisk måte å realisere dette på.

I en lagdelt løsning håndterer domenemodellen forretningsreglene, kontrollere koordinerer operasjoner, visningslaget står for presentasjon, og infrastrukturet tar seg av ting som filinnlasting og databasekommunikasjon. Denne oppdelingen gjør løsningen enklere å teste og gjør det mulig å videreutvikle én del uten å måtte endre hele systemet.

2.5 Enkapsulering og validering

Enkapsulering innebærer at intern tilstand skal beskyttes og kun eksponeres gjennom kontrollerte grensesnitt. Tett knyttet til dette er validering: domeneobjekter bør hindre at ugyldig tilstand oppstår allerede ved opprettelse eller endring.

En robust løsning sørger for at ugyldige verdier, som tomme navn, negative beløp, ugyldige priser eller ulovlige antall, avvises tidlig. Dette reduserer behovet for defensiv kode andre steder i systemet og styrker korrektheten i løsningen som helhet.

2.6 Immutabilitet og defensiv programmering

Immutabilitet kan gjøre objekter mer pålitelige og lettere å resonnerer om. Selv når full immutabilitet ikke er praktisk, kan defensiv programmering brukes for å beskytte systemets interne tilstand. Et typisk eksempel er å returnere umodifiserbare samlinger i stedet for direkte referanser til interne lister.

Dette er spesielt relevant i modell- og repositoryklasser, der eksponering av mutable datastrukturer ellers kan føre til at andre deler av systemet utilsiktet bryter med etablerte invariants.

2.7 Persistens og repository-design

Persistens handler om å lagre applikasjonens tilstand slik at den kan gjenopprettes senere. I programvareutvikling er det vanlig å kapsle dette inn bak en repository-abstraksjon, slik at lagringsteknologi ikke blir blandet inn i forretningslogikken.

Repository-basert persistens gir bedre modularitet fordi det isolerer valg av lagringsteknologi fra resten av applikasjonen. I dette prosjektet lagres spilltilstanden i SQLite, noe som gir en mer strukturert og realistisk løsning enn enklere filbaserte alternativer.

2.8 Automatisert testing

Automatisert testing er en sentral praksis for å validere oppførsel og redusere risikoen for regresjoner. Enhetstester er særlig nyttige når systemet inneholder forretningsregler, validering og beregninger som må forbli korrekte selv om koden videreutvikles.

En god teststrategi fokuserer på kode med høy verdi og høy risiko, som domeneklasser, beregninger, transaksjonslogikk og datainnlasting. Testing gjør det også tryggere å refaktorere, fordi utvikleren raskere kan oppdage om observerbar oppførsel endres utilsiktet.

3 METODE

*[I dette kapitlet skal dere beskrive hva som skal til for å kunne reprodusere resultatet dere har fått. I programvareutvikling koker det som oftest ned til **prosess/metodikk, og verktøy.**]*

3.1 Utviklingsprosess

[I dette kapitlet skal du fortelle hvilken prosess du/dere planla å følge. Den skal dekke prosessmodellen, hvorfor den ble valgt og hvordan den ble implementert.]

Har dere jobbet i gruppe, så si noe om hvordan dere planla å organisere arbeidet (hvor ofte planla dere å møtes å jobbe?)

Hvordan planlagt prosess faktisk fungerte og hvilke endringer dere eventuelt gjorde skal beskrives i resultat-kapitlet og drøftes under Drøfting.

*I en typisk mappe-oppgave der dere har jobbet gjennom flere stadier/deler av prosjektet, og fått tilbud om tilbakemelding, bør dette beskrives som en del av prosessen (altså **at** dere jobbet i f.eks. 3 deler med muntlig tilbakemelding etter hver del).]*

Siden dette prosjektet ble gjennomført som en samarbeidsbasert mappeoppgave, stilte utviklingsprosessen krav både til teknisk struktur og koordinering mellom gruppemedlemmer. Den valgte arbeidsformen kan best beskrives som iterativ og inkrementell. I stedet for å forsøke å designe og implementere hele systemet i én sammenhengende fase, ble løsningen utviklet stegvis, der grunnleggende funksjonalitet først ble etablert og deretter utvidet og forbedret.

En tidlig prioritet var å identifisere de viktigste domenebegrepene og etablere en arkitektur som kunne støtte videre vekst. Dette innebar blant annet å modellere spilleren, aksjene, porteføljen, transaksjoner og børsen, samtidig som det måtte tas stilling til hvordan brukergrensesnitt, domenelogikk og persistens skulle samhandle. Når denne grunnstrukturen var på plass, kunne videre utvikling skje i gjentatte runder med planlegging, implementasjon, testing og forbedring.

I en gruppe er en slik arbeidsform særlig nyttig fordi den gjør det mulig å dele opp arbeidet i tydelige ansvarsområder samtidig som man bevarer en helhetlig arkitektur. Ett gruppemedlem kan for eksempel arbeide med brukergrensesnitt, et annet med domenelogikk, og et tredje med persistens eller tester, så lenge grensene mellom komponentene er tydelige. Dette reduserer konflikter og gjør det enklere å arbeide parallelt.

Den iterative prosessen gjorde det også mulig å tilpasse løsningen underveis. Enkelte designvalg blir først virkelig forståelige når systemet begynner å ta form, og det var derfor naturlig å justere struktur og ansvar etter hvert som prosjektet utviklet seg. Dette er en arbeidsform som passer godt i et porteføljeprojekt, der innsikt og forståelse vokser gjennom implementasjonen.

3.2 Verktøy

[Beskriv verktøy du/dere har benyttet for å løse prosjektet. Lag gjerne en tabell med navn på verktøy, versjon og hva verktøyet er benyttet til. Få med **samtlige** verktøy (IDE, versjonskontroll, prosjektplanlegging/gjennomføring osv)]

Verktøy	Versjon	Hensikt
Java Development Kit (JDK)	25	Programmeringsspråk og runtime environment
Apache Maven	POM 4.0.0	Bygging av prosjekt, avhengigheter og testkjøring
Maven Compiler Plugin	3.14.1	Kompilering mot Java 25
Maven Surefire Plugin	3.5.4	Kjøre JUnit-tester
JavaFX Controls	25.0.1	Grafisk brukergrensesnitt
SQLite JDBC	3.45.1.0	Databasekommunikasjon for persistens
JUnit Jupiter	6.0.1	Automatisert testing
IntelliJ IDEA	[fyll inn ved behov]	Utviklingsmiljø
Git	[fyll inn ved behov]	Versjonskontroll
GitHub	[fyll inn ved behov]	Lagring og samarbeid
Draw.io / PlantUML	[fyll inn ved behov]	Diagrammer og visualisering

3.3 Bruk av KI verktøy

[I starten av rapporten skal dere fylle ut en **KI-deklarasjon**. Du/dere kan selv velge om dere vil beskrive **hva** og **hvordan** dere har benyttet KI-verktøy enten i skjemaet, eller her i dette kapittelet. Dersom dere velger å beskrive det her, så **henviser du/dere til dette kapittelet i KI-deklarasjons-skjemaet**. Beskriv også her **hvorfor** du/dere valgte å benytte KI-verktøy. Hva ønsket dere å oppnå?]

KI-baserte verktøy ble brukt i begrenset og støttende grad under arbeidet med prosjektet og rapporten. Formålet var ikke å erstatte egen forståelse eller selvstendig utviklingsarbeid, men å bistå med formulering, strukturering og språklig forbedring.

I forbindelse med rapportskrivning ble KI brukt til å formulere tekst, forbedre akademisk språk og bidra til tydeligere struktur i innholdet. Slik bruk kan være nyttig for å effektivisere skriveprosessen, men forutsetter at all tekst gjennomgås kritisk og kontrolleres opp mot faktisk kodebase, oppgavetekst og gruppens egne erfaringer.

[Dersom KI også ble brukt i andre deler av arbeidet, for eksempel til idéutvikling, språkvask, syntakshjelp eller forslag til tester, bør dette beskrives presist og ærlig. Det avgjørende er at den endelige besvarelsen fortsatt gjenspeiler gruppens egen forståelse og ansvar for både kode og rapport.]

4 RESULTAT

4.1 Teknisk Design

[Kapittelet om teknisk design beskriver det store bildet av valgt løsning. For et programvareutviklingsprosjekt vil dette vanligvis inneholde systemarkitekturen (klient-server, sky, databaser, tjenester, desktop-applikasjon osv.); både hvordan det ble løst, og, enda viktigere, hvorfor denne arkitektur ble valgt]

Den endelige løsningen er en desktop-applikasjon med grafisk brukergrensesnitt implementert i JavaFX. Arkitekturen er delt inn i flere samarbeidslag: modellklasser for kjernedomenet, kontrollere som kobler sammen tilstand og brukerinteraksjon, visningsklasser for presentasjon, samt infrastrukturlag for datainnlasting og persistens.

I sentrum av designet står selve aksjemarkedssimuleringen. Klassen **Exchange** representerer markedet og holder oversikt over tilgjengelige aksjer samt hvilken uke simuleringen befinner seg i. Hver **Stock** inneholder symbol, selskapsnavn og prishistorikk, noe som gjør det mulig å beregne både nåværende pris og utvikling over tid. Klassen **Player** representerer brukerens økonomiske tilstand, inkludert kontantbeholdning, portefølje og transaksjonsarkiv. **Portfolio** holder oversikt over eide aksjer, mens **Transaction** og underklassene **Purchase** og **Sale** modellerer gjennomførte handler.

En tydelig styrke ved arkitekturen er at domenelogikk ikke er plassert direkte i brukergrensesnittet. I stedet brukes kontrollere som **ExchangeController** og **ProfileController** til å koordinere samspillet mellom modell og visning. Dette gjør at visningslaget i større grad kan fokusere på presentasjon og hendelser, mens forretningsreglene forblir samlet i domenemodellen og kontrollerlaget.

Løsningen inneholder også et eget infrastrukturlag. Aksjedata lastes fra CSV gjennom **StockCsvLoader**, mens lagring og innlasting av spilltilstand håndteres via repository-abstraksjoner og en SQLite-basert implementasjon. Denne separasjonen gjør at lagringslogikk holdes adskilt fra handels- og domenelogikk, noe som styrker både vedlikeholdbarhet og utvidbarhet.

Samlet sett fremstår den tekniske løsningen som en lagdelt, objektorientert arkitektur der sentrale begreper er modellert eksplisitt, ansvar er relativt tydelig fordelt, og systemets viktigste operasjoner er organisert rundt meningsfulle abstraksjoner i stedet for prosedyrisk kode.

4.2 Implementasjon

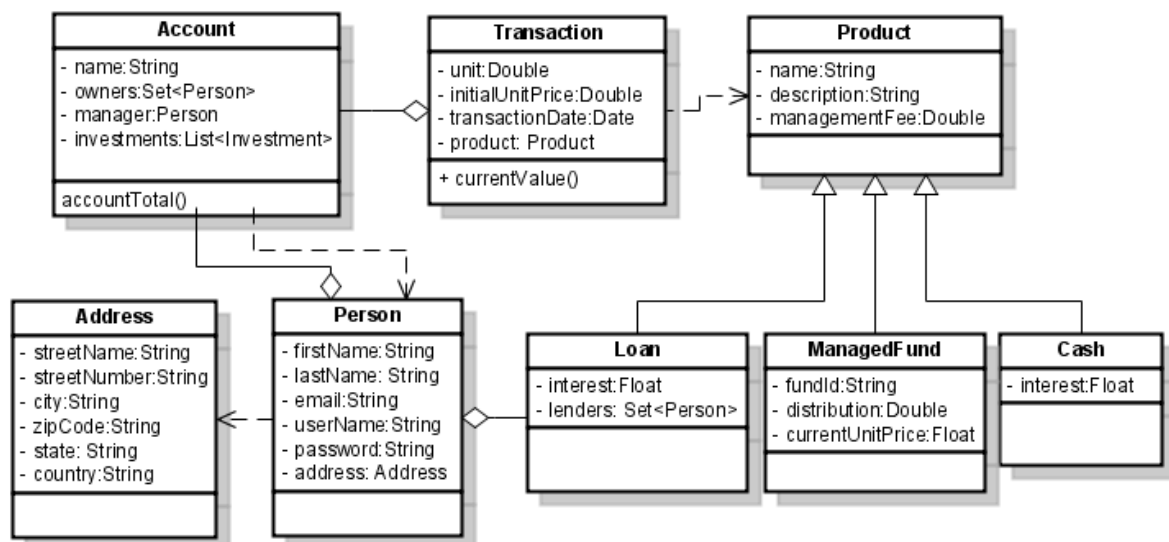
[Her beskriver du de tekniske detaljene til løsningen du har endt opp med. Hvilke eksterne biblioteker og rammeverk, APIer osv. har du/dere benyttet?

Vis med **klassediagrammer** hvordan klassene i løsningen din henger sammen. Husk at du kan vise flere klassediagrammer for å vise ulike sider ved løsningen (kun klasser som benyttes til brukerinteraksjon f.eks., eller kun klasser som utgjør forretningslogikken).

Bruk **aktivitetsdiagramm(er)** for å beskrive logikken/flyten i løsningen din. Fint også om du kan vise hvordan objekter av klassene dine **samhandler** for å løse **de mest sentrale** oppgavene i løsningen, da i form av **sekvensdiagramm(er)**.

Klassene dine bør beskrives i forhold til hvilken **rolle** og **ansvar** de har, men det er ikke nødvendig å beskrive hver enkelt metode eller felt til klassen. De står uansett dokumentert i Javadoc'en din.]

Implementasjonen er skrevet i Java og bygget med Maven. Byggoppsettet er konfigurert for Java 25 og benytter avhengigheter for JavaFX, SQLite JDBC og JUnit. Denne teknologistacken gjør det mulig å kombinere et moderne grafisk brukergrensesnitt med en mer strukturert persistensløsning enn ren filbasert lagring.



Figur 2 Klassediagram som viser...

Domenemodellen inneholder de viktigste begrepene i simuleringen:

- **Stock** lagrer ticker-symbol, selskapsnavn og historiske priser
- **Share** representerer en beholdning av én aksje med antall og kjøpspris
- **Portfolio** holder oversikt over brukerens eide aksjer
- **Player** representerer brukeren med navn, startkapital, nåværende saldo, portefølje og transaksjonshistorikk
- **Transaction** er en abstrakt basisklasse for handler, med **Purchase** og **Sale** som konkrete underklasser
- **TransactionArchive** lagrer historikk over gjennomførte handler
- **Exchange** administrerer markedet, aksjene, tidsutviklingen og transaksjonsutførelse
- **GameState** samler børss- og spillerdata for lagring

Beregninger knyttet til kjøp og salg er skilt ut i egne kalkulatorklasser. **PurchaseCalculator** beregner total kostnad for kjøp inkludert gebyr, mens **SaleCalculator** beregner netto utbetaling ved salg, inkludert gebyr og eventuell gevinstskatt. Dette gjør at finansiell logikk er isolert fra resten av transaksjonsgjennomføringen, noe som styrker både lesbarhet og testbarhet.

Kontrollerlaget fungerer som et bindeledd mellom modellen og brukergrensesnittet. **ExchangeController** eksponerer operasjoner som kjøp, salg, søk, tidsprogresjon og beregning av totalsummer, mens **ProfileController** henter ut mer sammensatte brukerdata som gevinst- og tapsoversikter. Dette gjør at grensesnittkoden slipper å forholde seg direkte til lavnivålogikk i domenemodellen.

Visningslaget er implementert med JavaFX og består blant annet av startside, lasteside, dashboard, kjøps- og salgsdialoger, samt profilsider. Applikasjonen starter i et eget startvindu og går videre til dashboardet når brukeren oppretter eller laster et spill. Dette gir en mer helhetlig brukeropplevelse enn en konsollbasert løsning og er i tråd med ambisjonsnivået man kan forvente i et prosjekt i Programmering 2.

Persistens er implementert gjennom **SqliteGameRepository**, som oppretter og håndterer en relasjonsdatabase med tabeller for lagrede spill, spillerdata, aksjer, prishistorikk, porteføljeandeler og transaksjonslogg. Dette er et sterkt valg fordi det demonstrerer mer avansert tilstandshåndtering enn en enkel tekst- eller JSON-basert løsning, og det bidrar til at systemet fremstår mer som en realistisk applikasjon.

Et implementasjonsmessig forhold som er verdt å nevne, er at repositoryet ser ut til å inneholde både `model`- og `domain`-varianter av enkelte komponenter. Dette kan tolkes som et resultat av iterativ refaktoring underveis i prosjektet. Det er i seg selv ikke nødvendigvis negativt, men det kan tyde på at løsningen fortsatt har rom for opprydding og konsolidering.

4.3 Testing

[Beskriv hvordan løsningen din har blitt testet, både i form av enhetstesting, men også brukertesting. Hvor mange brukere har testet løsningen? Hvordan ble brukertesting gjennomført? Vis også eventuelt resultatene fra brukertesting her. (svar på spørreskjema el.l.)]

Enhetstesting: si noe om hvilke klasser du/dere har valgt å skrive tester for, og hva strategien som dere/du har lagt opp til for å sikre best mulig testet kode.]

Prosjektet inneholder automatiserte tester skrevet med JUnit, og testene retter seg primært mot domenelogikk og støttekomponenter snarere enn direkte mot brukergrensesnittet. Dette er en hensiktsmessig strategi, siden den mest verdifulle automatiserte testingen vanligvis er den som verifiserer forretningsregler, beregninger og grensetilfeller i ikke-visuell kode.

Repositoryet inneholder tester for flere sentrale områder:

- **ExchangeTest** tester aksjeoppslag, kjøp, salg, søk og ukentlig progresjon
- **PortfolioTest** tester innlegging, fjerning, oppslag og defensiv oppførsel i porteføljen
- **TransactionTest** tester commit-logikken for kjøp og salg, inkludert feilsituasjoner
- **PurchaseCalculatorTest** og **SaleCalculatorTest** tester finansielle beregninger
- **StockTest** tester prishistorikk og avledede verdier
- **StockCsvLoaderTest** tester innlasting fra CSV, inkludert kommentarer, blanke linjer og feilformat
- **TransactionArchiveTest** tester transaksjonshistorikk
- **PlayerStatusTest** tester utvikling av spillerstatus basert på handelsaktivitet og verdiutvikling

En styrke ved testene er at de ikke bare fokuserer på vellykkede scenarier, men også verifiserer ugyldig input og feilsituasjoner. Det testes blant annet for ukjente aksjer, utilstrekkelige midler, salg av aksjer som ikke eies, og forsøk på å modifisere umodifiserbare samlinger. Dette styrker påstanden om at løsningen er utviklet med vekt på robusthet.

Samtidig ser testingen i hovedsak ut til å være konsentrert rundt modell- og infrastrukturlagene. Det er et godt utgangspunkt, men det gir også rom for ytterligere forbedringer gjennom flere integrasjonstester for persistensflyt eller kontrollerlogikk. Likevel viser den eksisterende testpakken en tydelig og relevant bruk av automatisert testing som passer godt i en besvarelse med høyt nivå.

4.4 Utrulling til sluttbruker (deployment)

[Her beskriver du hvordan programvaren din gjøres tilgjengelig for sluttbruker. Rulles den ut å en web-server? I så fall hvordan? Lages det en desktop-applikasjon som bruker kan dobbeltklikke på for å starte? Eller kjøres applikasjonen fra Maven (mvn javafx:run) Osv.]

Applikasjonen distribueres som en Java-basert desktop-applikasjon bygget med Maven. I praksis innebærer dette at brukeren må ha et kompatibelt Java-miljø tilgjengelig for å kunne kjøre programmet. README-filen i repositoryet viser at applikasjonen kan startes enten gjennom Maven eller direkte fra IDE, med JavaFX-applikasjonen som hovedinngang.

Rent teknisk består utrulling av å klonere prosjektet, la Maven hente nødvendige avhengigheter, og deretter starte programmet. Siden JavaFX og SQLite-avhengigheter er deklartert i pom.xml, håndteres oppsettet automatisk av byggverktøyet. Dette forenkler kjøringen for en bruker som allerede har riktig Java-oppsett.

Lagrede spill lagres lokalt ved hjelp av SQLite. Dette betyr at brukerdatabaser blir liggende på maskinen der applikasjonen kjøres, og at løsningen ikke er avhengig av noen ekstern backend-tjeneste. Innenfor rammene av et emneprosjekt er dette en hensiktsmessig distribusjonsmodell, fordi den holder kompleksiteten nede samtidig som den viser at systemet håndterer varig tilstand.

5 DRØFTING

5.1 Drøfting av løsning/design

[Her oppsummerer du/dere oppgaven. Hvor langt kom du/dere (resultat)? Hva fikk du/dere ikke gjort i forhold til oppgaveteksten? Hva var de store utfordringene/problemene du/dere møtte, etc..]

*Spesielt viktig er det å **drøfte din egen løsning i forhold til det du har lært om gode prinsipper for design av programvare** (robust kode, kodestil, designprinsipper osv) som beskrevet i teori-kapittelet. I en godt skrevet rapport, er det ingen teorier som beskrives under teori-kapittelet som ikke drøftes under drøfting-kapittelet.*

*Husk å være **konkret**: Det holder ikke å skrive «Jeg/vi har designet en løsning som er i tråd med prinsippene om coupling og cohesion». Du/dere må «**bevise**» **hvorfor** dere kan påstå dette. Altså hente eksempler fra egen kode som underbygger teoriene om god design: «I klassen har vi valgt å returnere fra metoden.... Dette bidrar til lav kobling....»*

Her bør man også gjøre seg tanker rundt kvaliteten av det arbeidet som er nedlagt.

Er de kildene du/dere bruker pålitelige, er det sprik mellom forskjellige kilder (og i så fall hvorfor), er det andre forhold som kan være med å gjøre noen av de vurderinger og valg du/dere har gjort usikre?]

Den endelige løsningen lykkes i å implementere et sammenhengende aksjemarkedssimuleringsspill med grafisk brukergrensesnitt, domenemodell, persistens og testing. Sett i lys av forventningene til et prosjekt i Programmering 2 er dette et sterkt resultat, fordi løsningen kombinerer flere sentrale sider ved programvareutvikling i ett integrert system.

En av de tydeligste styrkene i designet er den eksplisitte domenemodelleringen. Klasser som **Stock**, **Share**, **Player**, **Portfolio**, **Exchange** og **Transaction** gir kodebasen en klar begrepsmessig struktur. Dette gjør systemet lettere å forstå, fordi de viktigste abstraksjonene i koden samsvarer godt med problemdomenet. I stedet for å plassere all logikk i én stor kontrollør eller i brukergrensesnittet, er ansvaret fordelt på flere klasser med tydeligere roller.

Løsningen demonstrerer også lav kobling og relativt høy samhörighet i viktige deler av kodebasen. Beregninger knyttet til kjøp og salg er for eksempel skilt ut i **PurchaseCalculator** og **SaleCalculator**, i stedet for å være spredt i transaksjons- eller visningslogikk. På samme måte er persistens skilt ut i egne repository- og lagringsklasser, i stedet for å være tett koblet til domenemodellen. Dette gjør det enklere å endre lagringsstrategi eller beregningsregler uten at hele systemet må bygges om.

Enkapsulering er også tydelig i flere av modellklassene gjennom konstruktørvalidering og begrenset direkte tilgang til intern tilstand. **Player**, **Share** og **Stock** validerer input ved opprettelse, og klasser som **Portfolio** og **Stock** eksponerer umodifiserbare samlinger i stedet for rå mutable lister. Dette støtter defensiv programmering og reduserer risikoen for at intern tilstand blir ødelagt av ekstern kode.

Et annet sterkt designvalg er bruken av SQLite for persistens. Sammenlignet med enklere serialisering gir en repository-løsning med relasjonelle tabeller en mer strukturert og skalerbar måte å lagre spilltilstand på. Den viser også en mer moden forståelse av persistens, særlig fordi systemet lagrer spillerdata, aksjer, prishistorikk, porteføljeinnhold og transaksjonslogg separat.

Brukergrensesnittet representerer også en tydelig forbedring sammenlignet med en konsollbasert løsning. JavaFX-baserte sider, paneler og dialoger gir applikasjonen et mer

ferdig produktpreg og viser et høyere ambisjonsnivå enn en minimal implementasjon. For et prosjekt i Programmering 2 er dette relevant fordi det viser ikke bare evne til å programmere funksjonalitet, men også til å utvikle en mer helhetlig applikasjon.

Det finnes likevel noen svakheter og begrensninger som bør drøftes. Repositoryet ser ut til å inneholde overlappende pakkestrukturer, blant annet både `model` og `domain` for enkelte deler av løsningen. Dette kan tyde på pågående refaktorering eller ufullstendig konsolidering. Selv om iterativ forbedring i seg selv er positivt, kan slike duplikater gjøre løsningen vanskeligere å navigere i og vedlikeholde hvis de blir stående.

Et annet punkt er at enkelte implementasjonsvalg i større grad ser ut til å være drevet av spesifikasjonen enn av ideell domenemodellering. Bruken av `BigDecimal` som datatype for aksjeantall kommenteres for eksempel direkte i koden som et litt unaturlig valg. Dette er forståelig dersom det følger av oppgaveteksten, men viser samtidig spenningen mellom å følge spesifikasjonen nøyaktig og å lage den mest realistiske modellen.

Helhetsinntrykket er likevel at løsningen er teknisk sterk. Den balanserer struktur, funksjonalitet, persistens, testing og brukerinteraksjon på en gjennomtenkt måte. Designet fremstår ikke bare som funksjonelt, men også som faglig begrunnet og godt nok strukturert til å støtte en høy vurdering.

5.2 Drøfting av prosess

[Fulgte dere prosessen som dere planla (og beskrev under kapittelet «Metode»)? Var det lurt, eller ikke? Hva fungerte bra hva fungerte mindre bra? Hva ville du/dere ha gjort annerledes neste gang?]

Den iterative utviklingsprosessen fremstår som et godt valg for dette prosjektet. Et prosjekt av denne typen inneholder flere ulike hensyn – domenemodellering, beregninger, brukergrensesnitt, persistens og testing – og disse er vanskelige å ferdigstille korrekt i én enkelt designfase. En iterativ tilnærming gjør det mulig å etablere en fungerende grunnløsning og deretter forbedre både struktur og funksjonalitet over tid.

I et gruppeprosjekt er denne arbeidsformen særlig verdifull fordi den muliggjør parallelt arbeid. Når arkitekturen først er avklart, kan ett gruppemedlem arbeide med brukergrensesnitt, et annet med persistens og et tredje med tester eller domenelogikk. Dette forutsetter tydelige grensesnitt og en viss enighet om struktur, men når det fungerer godt, øker det både fremdrift og fleksibilitet.

Repositoryets struktur tyder også på at prosessen innebar forbedring og omorganisering underveis. Det er naturlig i programvareutvikling. Ofte blir det først tydelig etter at implementasjonen er påbegynt at enkelte ansvarsområder bør flyttes, abstraheres bedre eller organiseres annerledes. I så måte bør refaktorering ikke tolkes som tegn på svak planlegging, men snarere som en del av det å forbedre løsningen gjennom innsikt opparbeidet underveis.

Dersom noe kunne vært gjort annerledes, er det sannsynligvis knyttet til den avsluttende oppryddingen i kodebasen. Arkitektonisk utvikling og omstrukturering er positivt, men i en sluttinnlevering er det en fordel om repositoryet kommuniserer én tydelig og konsistent struktur. En mer ryddig sluttversjon ville styrket helhetsinntrykket ytterligere.

5.3 Drøfting av bruken av KI-verktøy

[Dersom du/dere benyttet KI-verktøy i denne oppgaven, drøft kort erfaringene dine/deres. Hva var KI-verktøyene nyttige for å løse? Hvilke svakheter oppdaget du/dere?]

Har du/dere ikke benyttet KI-verktøy dropper du dette kapittelet.]

Dersom KI-verktøy har vært brukt i prosjektet og rapportskrivningen, avhenger nytteverdien i stor grad av hvordan de har blitt brukt. Innen programvareutvikling kan KI være nyttig til språkvask, dokumentasjon, alternative formuleringer, strukturering av innhold og forslag til testscenarier. Slik bruk kan bidra til effektivitet uten nødvendigvis å redusere den faglige egeninnsatsen.

Samtidig har KI klare svakheter. Verktøyene kan produsere tekst og kode som virker overbevisende, men som ikke nødvendigvis stemmer med faktisk implementasjon eller oppgavetekst. KI-genererte forslag kan derfor ikke behandles som autoritative, men må alltid kontrolleres opp mot kodebasen, kravene i oppgaven og utviklernes egen forståelse.

I sammenheng med denne rapporten er den mest forsvarlige bruken av KI å bruke den som skrivehjelp, ikke som erstatning for analyse. Så lenge alle tekniske påstander etterprøves mot det faktiske prosjektet og gruppen selv står ansvarlig for innholdet, kan begrenset bruk av KI være forenlig med god faglig praksis.

6 KONKLUSJON - ERFARING

[Overbevisninger /erfaring som en er kommet fram til på grunnlag av det presenterte materialet.

Fikk du realisert hele problemstillingen fra kapittel «0 Dette prosjektet omhandler utviklingen av et aksjemarkedssimuleringsspill kalt Millions, utviklet som en del av mappeoppgaven i emnet IDATT2003 Programmering 2. Formålet med applikasjonen er å simulere handel i et forenklet aksjemarked, der brukeren kan kjøpe og selge aksjer, følge utviklingen i egen portefølje, se transaksjonshistorikk og observere hvordan aksjepriser endrer seg over tid.

I motsetning til en mindre, rent algoritmisk oppgave, krevde dette prosjektet utvikling av en helhetlig programvareløsning med både grafisk brukergrensesnitt og en intern arkitektur som håndterer domenelogikk, validering, lagring og administrasjon av tilstand. Prosjektet kombinerer dermed praktisk programvareutvikling med objektorientert design, brukerinteraksjon og fokus på vedlikeholdbar kode.

Den endelige løsningen er implementert som en Java-basert desktop-applikasjon med JavaFX som brukergrensesnitt. Programmet laster aksjedata fra CSV-filer, simulerer ukentlige endringer i markedet, lar brukeren gjennomføre handler, og lagrer spilltilstanden ved hjelp av en SQLite-basert persistensløsning. På denne måten går prosjektet utover en enkel prototype og fremstår som et mer helhetlig programvareprodukt.

- *Kravspesifikasjon»?*
- *Hva ville du ha gjort annerledes dersom du kunne begynn på nytt?*
- *Hva slags begrensninger kan en forvente når en bruker løsningen?*
- *Hva skal tas opp i fremtidige arbeid dersom du eller noen andre ville ha tatt utvikling videre?]*

Prosjektet resulterte i et komplett og relativt godt strukturert aksjemarkedssimuleringsspill som demonstrerer relevant kompetanse innen objektorientert programmering, programvarearkitektur, persistens, testing og grafisk brukergrensesnittutvikling. Applikasjonen lar brukeren opprette eller laste inn et spill, søke etter og handle aksjer, følge prisutvikling over tid, holde oversikt over porteføljen og lagre fremgangen lokalt.

Fra et programvareteknisk perspektiv er de sterkeste sidene ved prosjektet den tydelige domenemodellen, separasjonen mellom brukergrensesnitt, kontrollere og infrastruktur, bruken av automatiserte tester for sentral logikk, og implementasjonen av strukturert persistens med SQLite. Disse valgene gjør løsningen mer vedlikeholdbar og mer representativ for reell applikasjonsutvikling enn en minimal kursprototype.

Dersom prosjektet skulle videreutvikles, finnes det flere naturlige forbedringsmuligheter. Arkitekturen kunne vært ytterligere konsolidert der refaktorering har etterlatt overlappende strukturer. Persistensflyten kunne vært styrket gjennom flere integrasjonstester. Brukergrensesnittet kunne også blitt utvidet med rikere visualiseringer, mer avanserte filtre og bedre porteføljeanalyse.

Til tross for disse mulighetene fremstår prosjektet allerede som et sterkt resultat. Det oppfyller ikke bare sentrale funksjonelle mål, men viser også refleksjon rundt designprinsipper, implementasjonskvalitet og utviklingsprosess. Derfor gir prosjektet et godt grunnlag for en rapport på høyt nivå og støtter argumentet om at løsningen kan vurderes som en sterk besvarelse.

REFERANSER

[Forfatter, årstall, tittel på bok eller artikkel, navn på tidsskrift eller forlag/utgiver, nr. eller dato for tidsskrift, sted som det vises til eller refereres fra i oppgaven.]

Konkret for programmeringsemner: Regner med at du/dere kommer til å måtte slå opp litt i læreboka, så den er en innlysende referanse. Dersom du/dere i tillegg benytter internett, så list URL'er til sidene du/dere har benyttet.

Her er en god guide til hvordan oppgi referanser og hvordan referere til de (benyttes av IEEE): <https://www.bath.ac.uk/publications/library-guides-to-citing-referencing/attachments/ieee-style-guide.pdf>]

- [1] Gamma, E., Helm, R., Johnson, R., and Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [2] Martin, R. C. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
- [3] Oracle. *The Java Tutorials*.
- [4] OpenJFX Documentation.
- [5] SQLite Documentation.
- [6] Fowler, M. *Patterns of Enterprise Application Architecture*.
- [7] Wikipedia contributors. "Coupling (computer programming)."
- [8] Wikipedia contributors. "Cohesion (computer science)."
- [9] Wikipedia contributors. "Model-view-controller."

VEDLEGG

[Materiell som er utarbeidet eller innsamlet i tilknytning til rapporten, men som det ikke er naturlig eller hensiktsmessig å ta inn i hoveddelen, som feks brukerveiledning, skal tas inn som vedlegg.

Vedleggene skal være nummererte og ha en overskrift.

Har du/dere ingen vedlegg, så droppes dette kapittelet.]